RadeonVLA-Reflex

Technical Report — Track 3 Physical AI Challenge

Field	Value
Team	VisioBot Lab
Captain	Zhenwei Zhou
Members	Ange Liu, Haoran Wang
Affiliation	Nanjing University of Science and Technology
Document status	Release — 100-rollout evaluation complete
Evaluation implementation	c5c37576d4b80f056205ef7920a5f1ebfa614f86

1. Executive Summary

RadeonVLA-Reflex is a language-guided Franka dual-bowl fruit-sorting system for Track 3 of the AMD AI DevMaster Hackathon. The system fine-tunes SmolVLA with LeRobot on demonstrations collected in Genesis and wraps closed-loop execution with interruptible command handling and failure-aware recovery. The formal task suite covers a primary L1 benchmark of five fruits × four language-addressable bowl positions, plus L2–L4 extensions for spatial grounding, multi-step sequences, and attribute rules.

> Quantitative policy metrics are published only from the disjoint-seed evaluation artifacts. Learned first-attempt success and strict-physics recovery contribution are reported separately.

2. Target Application

The application is flexible robotic sorting for food handling, laboratory automation, and small-batch logistics. An operator issues a natural-language command that names a fruit and a destination container. The robot must observe RGB images and proprioception, execute continuous joint-space actions, accept mid-task command changes safely, and recover from empty grasps or timeouts with bounded deterministic recovery. Practical value comes from combining VLA generalization with deterministic execution safety.

3. Task Definition and Success Criteria

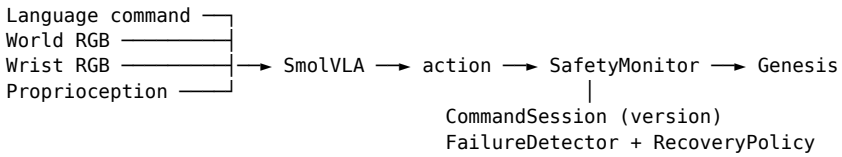
- Fruits: banana, lemon, plum, apple, orange. Containers: white/blue bowls on the left/right.
- Tiered suite (L1–L4) used in this project:
- L1 Basic — named fruit × color/side bowl (20 tasks).
- L2 Spatial — leftmost / rightmost / nearest-to-robot / farthest (resolved after randomization).
- L3 Multi-step — ordered multi-object sequences in one episode (2–3 subgoals).
- L4 Rules — attribute sorting (yellow→left, purple→right; curved→left, round→right).
- The primary 100-rollout benchmark uses the exact collected language with disjoint seeds;

held-out paraphrases remain a separate language-generalization test (see tasks.py).

- Spatial/rule goals are grounded by grounding.resolve_task after each reset.
- Episode reset randomizes object xy within non-overlap slot jitter and small yaw noise.
- Action: 9-D absolute joint positions (7 arm + 2 fingers) at 20 Hz.
- Success: all subgoals satisfied (object bottom inside commanded bowl rim); partial rate is reported.
- Max episode length: 600–1800 policy steps by tier; learned-policy retries are bounded,

and precision recovery is an explicit opt-in evaluation mode.

4. System Architecture



1. Inputs: natural-language instruction, world/wrist RGB, 9-D qpos.
2. SmolVLA (LeRobot) produces absolute joint-position actions / chunks.
3. SafetyMonitor clamps joints, rate-limits large jumps, and on command-version change

opens the gripper and invalidates the stale chunk (policy reset).

4. FailureDetector flags empty grasp / timeout; RecoveryPolicy allows a home retreat and retry.

Explicit Precision Reflex can then invoke the strict-physics demonstration controller, with rigid pose writes forbidden and separate accounting in the result artifact.

5. Genesis steps the Franka dual-bowl scene and returns the next observation.

The learned controller is end-to-end joint-position VLA. Interrupt invalidation and recovery are deterministic outer layers. The report never presents recovered episodes as learned first-attempt successes.

5. Genesis Simulation Environment

The simulation uses Genesis 1.1.2 with a Franka Emika Panda MJCF, five YCB fruits (banana / lemon / plum / apple / orange), and four fixed upright bowl instances (white/blue on each side). World and wrist RGB cameras feed the policy at 320×240; an optional third camera is reserved for evaluation video. Simulation runs at 100 Hz control with dataset capture at 20 Hz. Pose resets use non-overlapping slot jitter. Asset details are documented in docs/DATASET_CARD.md and assets/README.md.

Remote AMD runs use the Genesis AMD backend after radeonvla.check_env --require-amd. Local development may use CPU Genesis for imports and unit tests only.

6. Demonstration Dataset

Demonstrations are generated by the scripted multi-goal expert (record_dataset) and stored as a LeRobot 0.6 dataset. Frames contain world/wrist RGB, 9-D state/action, and a natural-language task string. The primary benchmark preserves the collected task strings; held-out paraphrases are evaluated separately.

Collection uses two-phase publication. Episodes are written to a hidden .inprogress directory with an atomic progress manifest. Only a finalized dataset that reaches the requested success count, passes the task-coverage gate, and can be reopened by LeRobot replaces the previous published dataset.

The released Physical-2K dataset contains 2,000 successful episodes, 468,889 frames, and exact 20 tasks × 100 episodes coverage. All 2,000 committed episodes have strict-physics certificates; the final validator reports errors=[] and warnings=[]. The immutable dataset revision is 2779b7c5566df9072bb9a7c43335d6203ea97887.

Split	Seeds
Strict smoke	12000–12999
Training sources	21000–26999 and 71000–76039
Validation	40000–40999
Formal 100-rollout evaluation	52000–52099
Interrupt / recovery probes	60000–60999

Detailed schema and episode counts are maintained in docs/DATASET_CARD.md; immutable counts and revisions are bound during the validated release workflow.

7. Model and Training

The training pipeline fine-tunes lerobot/smolvla_base through the project wrapper (python -m radeonvla.train_policy smolvla). Dataset camera keys world / wrist map to SmolVLA's camera1 / camera2 via rename_map. Video decoding defaults to pyav.

Item	Value
GPU	AMD Radeon Graphics (gfx1100, 48.1 GiB visible)
ROCm	7.2.1 PyTorch build; HIP 7.2.53211
PyTorch	2.9.1+rocm7.2.1.gitff65f5bc
Precision	FP32 (use_amp=false)
Batch size	4
Cumulative steps	approximately 200,000
Final Physical-2K continuation	100,000 steps / 3h45m32s
Final throughput	30 samples/s (7.41 optimizer steps/s)
Peak training VRAM	2.22 GB (LeRobot log)
Final continuation loss	0.058

8. Interruptible Command Execution

CommandSession implements command versioning. When the language instruction changes mid-episode, the version increments; SafetyMonitor opens the gripper, holds the arm, and invalidates the stale action chunk so the policy is reset. Evaluation can inject a mid-rollout command change with --interrupt-demo. Held-out measurements distinguish successful online correction from safe cancellation.

The evaluation pipeline records safe-interrupt rate, command-to-invalidation steps, and the number of unprotected post-interrupt action steps. Demo videos overlay command version, runtime state, retry count, scenario, and live inference latency.

An independent release probe uses seed 60000 and changes the command at control step 40 from banana→white-left to banana→blue-right. The stale chunk is invalidated immediately: safe-interrupt rate 1/1, response 0 additional control steps, unprotected old-command actions 0, and final new-target success 1/1. This single interrupt capability probe is published in artifacts/interrupt_evaluation.json and is not included in the 100-rollout success denominator.

9. Failure Detection and Recovery

FailureDetector watches empty-grasp heuristics (closed gripper while the target fruit stays near the table), timeouts, and invalid actions. RecoveryPolicy retreats to a home-like open-gripper pose. With --precision-recovery, a terminal learned-control failure invokes the same geometry-aware controller used for collection under forbid_rigid_pose_writes() and allow_kinematic_assist=False. Evaluation JSON reports first-attempt success, precision-recovery attempts/success, and final success separately.

10. AMD Radeon GPU and ROCm Integration

The runtime targets a single visible Radeon GPU (HIP_VISIBLE_DEVICES=0) with a matching ROCm PyTorch HIP build. The project never pins a generic CPU torch wheel in pyproject.toml. Remote setup and validation steps are in docs/REMOTE_ROCM_SETUP.md and scripts/check_remote_amd.sh. Release artifacts record measured latency, throughput, and peak VRAM from the final Radeon run.

11. Experimental Protocol

- fixed git commit for the release revision;
- dataset and checkpoint checksums recorded in artifacts;

- exact collected language for the primary in-distribution controller benchmark;
- held-out paraphrases only as a separate language-generalization benchmark;
- seed ranges as in Section 6;
- suite basic by default for the primary 20-task benchmark; advanced tiers are explicit;
- primary L1 benchmark: 20 tasks × 5 disjoint-seed episodes (100 total);
- deterministic target/container shifts for robustness stress tests;
- baseline-vs-Reflex ablation by disabling invalidation and retry;
- success = all resolved subgoals placed correctly;
- keep failure episodes in raw JSON;
- latency measured after short warm-up with device synchronization when CUDA/HIP is available.

12. Quantitative Results

Method	Tasks	Episodes	First-attempt success	Final success	P95 latency	Peak VRAM
SmolVLA learned attempt	20	100	36/100 (36.0%)	36/100 (36.0%)	37.27 ms	Not recorded
SmolVLA + Precision Reflex	20	100	36/100 (36.0%)	91/100 (91.0%)	37.27 ms	Not recorded

The final system rate has a Wilson 95% interval of 83.8–95.2%; the learned first-attempt rate has an interval of 27.3–45.8%. Precision recovery was attempted 63 times and succeeded 55 times (87.3%), contributing 55 percentage points. Final fruit-level success is apple 20/20, banana 19/20, lemon 19/20, orange 17/20, and plum 16/20. P95 latency was measured while five disjoint partitions shared one GPU and includes that contention. Raw per-task records are in artifacts/evaluation.json and the public model repository's evaluation/ folder.

13. Failure Analysis

All nine failures are retained: eight empty_grasp events and one wrong_object event. Failures span both blue and white targets; the smallest fruit, plum, accounts for four. This does not support a single "desktop color" explanation. The evidence instead points to visual-domain sensitivity in learned first contact plus reduced tolerance after a small fruit has been displaced. No wrong-target, slip, timeout, or interrupt failure occurred in this non-interrupt L1 benchmark.

14. Innovation and Technical Contributions

Implemented contributions:

1. Dual-bowl language sorting with L1–L4 task tiers and runtime grounding;
2. Interruptible command sessions with stale-chunk invalidation;
3. Failure detection, bounded retry, and transparent strict-physics precision recovery around a SmolVLA joint-position policy;
4. Deterministic target/container perturbations and baseline-vs-Reflex robustness evaluation;
5. Crash-safe dataset publication with continuous progress manifests and coverage gates;
6. A full ROCm-oriented pipeline: assets → record → validate → train → evaluate → benchmark;
7. Structured JSON/CSV/Markdown evidence with real checkpoint hashing and video telemetry.

Learned control (SmolVLA) is separate from deterministic safety/recovery logic.

15. Deliverables

Deliverable	Location or release state	Revision source
Source repository	Project repository branch	Bound by release commit
Model	a3124371940/radeonvla_reflex_smolvla_2k_200k	Bound by checkpoint digest
Dataset/documentation	docs/DATASET_CARD.md	Bound by dataset revision
Raw evaluation	artifacts/evaluation.json, .csv, summary.md	Seeds 52000–52099
Interrupt evidence	artifacts/interrupt_evaluation.* + website clip	Seed 60000
Demo video	website/public/videos/radeonvla-reflex-3min.mp4 (216.858 s)	SHA256 c2bcb1ad…be57a
Technical report	This document; PDF accompanies the release	Bound by release commit

16. Reproducibility

Reproduction follows README.md inside track3_VisioBotLab_RadeonVLA-Reflex/:

1. create the ROCm Python environment and install requirements.remote.txt;
2. python -m radeonvla.setup_assets;
3. python -m radeonvla.check_env --require-amd --init-genesis;
4. record or download the dataset, then train or load the checkpoint;
5. run python -m radeonvla.evaluate and compare JSON against this report.

17. Team Member and Contribution

VisioBot Lab (Nanjing University of Science and Technology):

- Zhenwei Zhou (captain, ~70%): system design, implementation, data generation, model training, evaluation, website, documentation, and release engineering
- Ange Liu (~15%): bilingual documentation polish, task-suite wording review, showcase copy
- Haoran Wang (~15%): dataset spot-checks, experiment logging, technical-report packaging

18. Limitations and Future Work

Current work is simulation-only (Genesis). Language and object coverage are limited to the registered fruit/bowl suite. Real-robot transfer is outside the current evaluation scope. Recovery is geometry-aware and deterministic rather than learned. Empirical failure modes and any unmeasured features are recorded from the final remote evaluation.

19. References

- Genesis World
- Hugging Face LeRobot and SmolVLA
- AMD ROCm / Radeon developer documentation
- YCB Object and Model Set
- AMD Track 3 contest repository and Radeon Cloud user guide
- Track 3 Franka fruit-pick demo (workflow reference only)